

CYBR 437: Secure Coding

Intro to GDB

Instructor: Abinash Borah

Outline

- Intro to GDB
- Walkthrough with GDB

GDB: GNU Debugger

- GDB is a powerful command-line debugging tool used mainly for C, C++, and other compiled languages (Fortran, Rust through extensions, etc.)
- We can run programs step-by-step, set breakpoints to pause execution, and inspect variables and memory to understand the program behavior at different stages
- By doing this, we can identify and fix bugs or crashes like segmentation faults
- GDB also supports remote debugging, which allows debugging programs running on different machines

Why use GDB?

- See exactly where the program stops working
- Inspect values of variables at any moment
- Debug segmentation faults and memory errors
- Work efficiently without print statements

How GDB Works

- GDB operates on executable files, which are binary files produced by the compilation process
- Uses debugging symbols from compiled program
- Interacts with machine instructions
- Allows stepping through lines or CPU instructions
- Allows modifying and examining program state as it runs

Typical Workflow

1. Compile with debug symbols

e.g., `gcc -g program1.c -o program1`

2. Launch GDB

e.g., `gdb ./program1`

3. Set a breakpoint

e.g., `(gdb) break main`

4. Run the program

e.g., `(gdb) run`

5. Inspect variables

e.g., `(gdb) print x`

6. Step through lines

e.g., `(gdb) next`